Name: V. Aravind
Reg no: 192321119
Course code: CSA1407
Course Name: Compiler Design
Date: 01/08/26

Short Answer Test

61/100

(1)

$E \rightarrow E+E/id$

$E \rightarrow E+E$

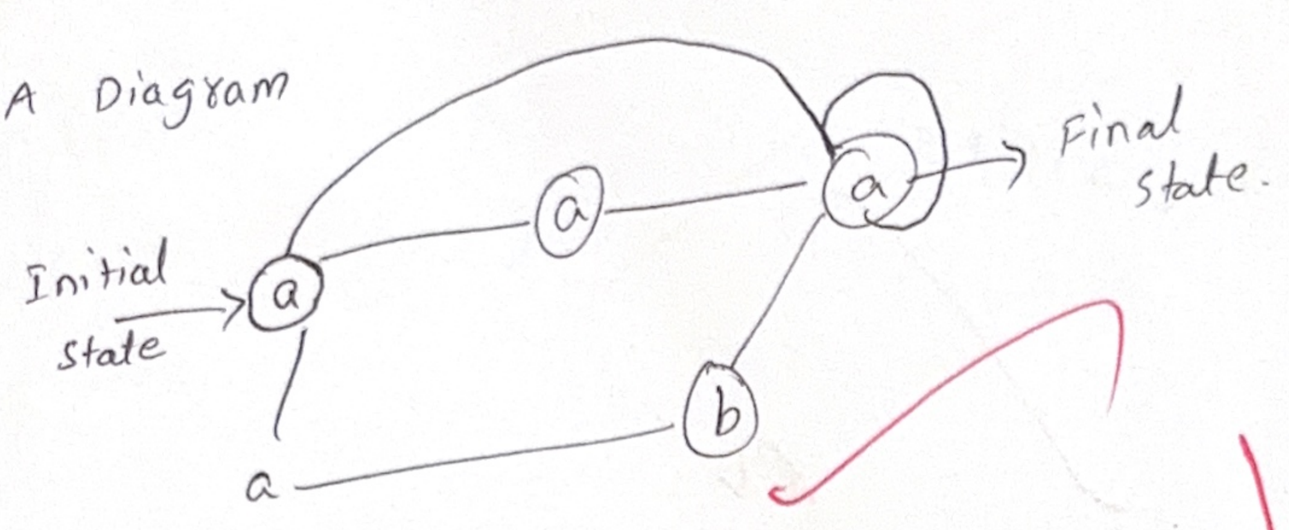
$E \rightarrow id$

(2)

a)

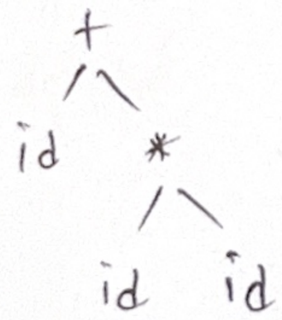
$a\,(a|b)^*\,a.$

NFA Diagram



(3)

$id + id * id$



(4)

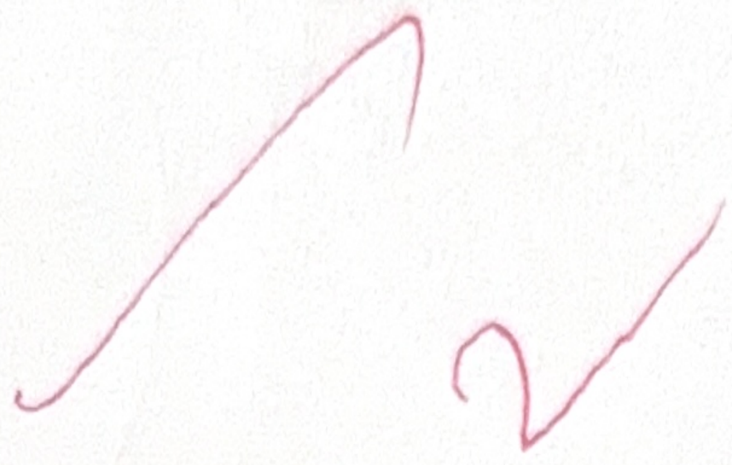
$A \to A\alpha / \beta$

$A \to \beta A'$

$A' \to \alpha A'$

$A' \to \varepsilon$

$A' \to \alpha A' / \varepsilon$



(5)

Given

$S \to iEtS / iEtScS / a$

$C \to b$

$A \to \alpha\beta / \alpha\gamma$

After left factoring

$A \to \alpha A$

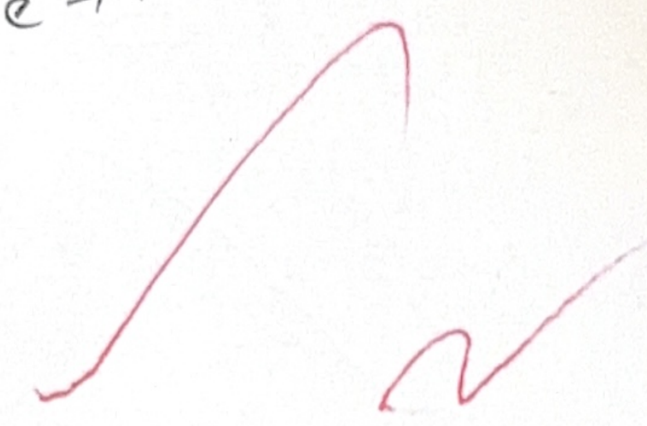
$A' \to \beta / \gamma$

$S \to ictS\ S'$

$S' \to eS / \varepsilon$

$C \to b$ //

$e \to iEtS$



(6) Given

$S \to aBDb$

$B \to cC$

$C \to bC / \varepsilon$

$D \to EF$

$E \to g / \varepsilon$

$F \to f / \varepsilon$

First

First {S} = {a}

First {B} = {c}

First {C} = {b, ε}

First {D} = {g, ε}

First {E} = {g, f, ε}

first {F} = {f, ε}

Follow

follow (S) = {$}

follow (B) = {g, f, $}

follow (C) = {g, f, $}

follow (D) = {g, f, $}

follow (E) = {g, f, $}

follow (F) = {g, f, $}

12

7)

Given grammar

S → Sa | Sb | $cd

S → Sa

S → Sb

S → cd

(8)

$E \to TE', E' \to +TE' / \varepsilon$ find First(E').

$E \to TE'$

$E' \to +TE' / \varepsilon$

First $(E') = \{+, \varepsilon\}$

(9)

$S \to (S)S / \varepsilon$

(10)

There are 4 types of actions:

1. Shift : Buffer on the stack.
2. Reduce : Production rule on the stack.
3. Accept : The symbol '$' the i/p string is accepted
4. Error : shift (or) reduce error.

(11)

Grammar:

$S' \to S$

$S \to CC$

$C \to cC / d$

item: $[S' \to .S]$

closure

$S' \to .S$

$S \to .CC$

$C \to .cC / .d$

9) $S \to AB$ First(S) = { a, b }

$A \to a$ First (A) = { a }

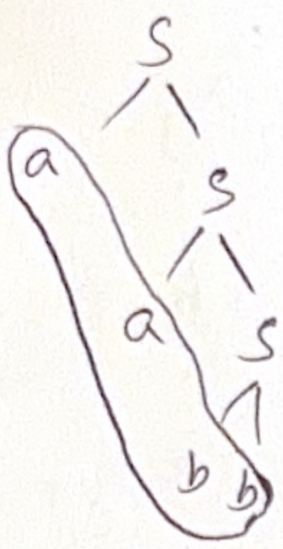
$B \to b$ First (B) = { b }

8) $S \to aSb \mid ab$.

key = a a b b

$S \to aSb$

$S \to ab$



2

3)

$S \to A$

$A \to aB / Ad$

$B \to b$

$C \to g$

First

First (S) = { a }

First (A) = { a } , first (A') = { d, c }

First (B) = { b }

First (C) = { g }

Follow

follow (S) = { $ }

follow (A) = { $ } , follow (A') = { $ }

follow (B) = { d, $ }

follow (C) = { d, $ }

2

(15)

given grammar

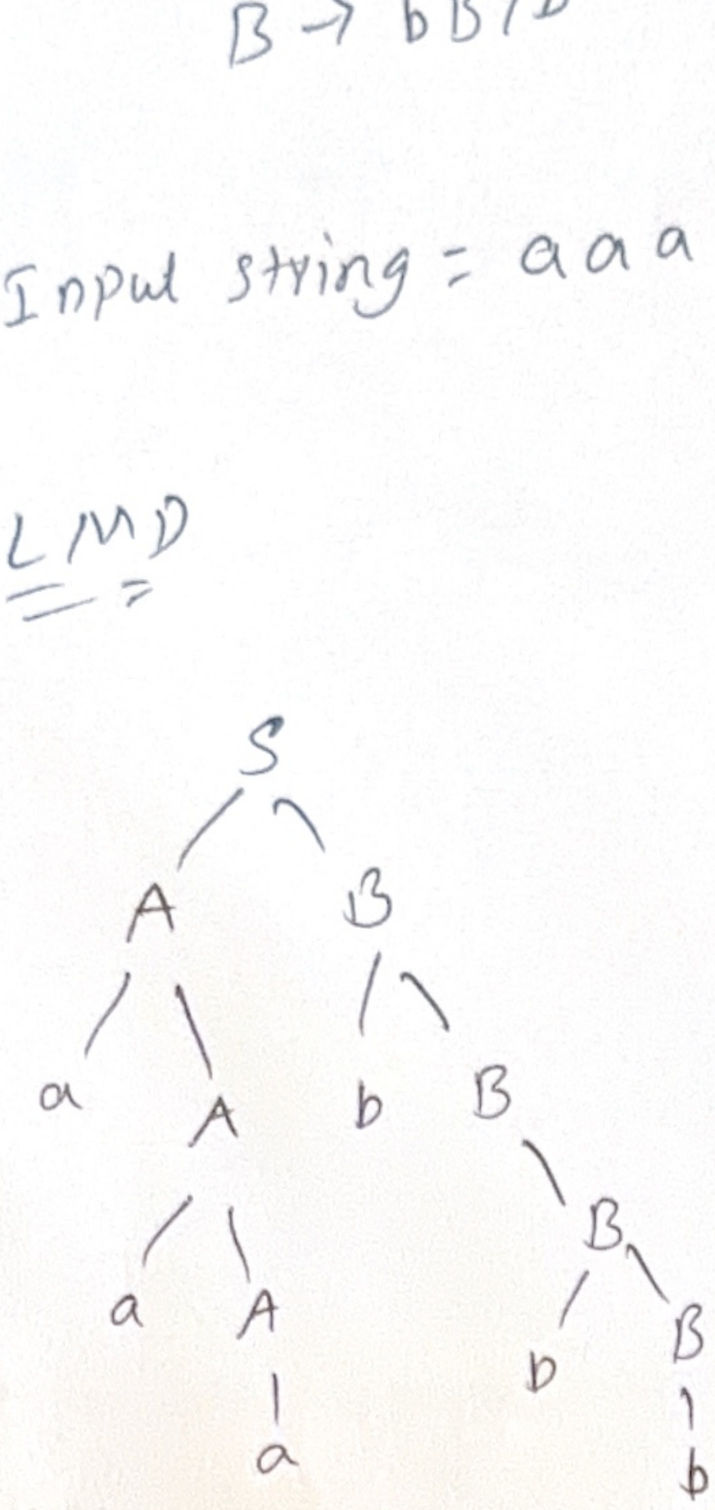
$S \rightarrow AB$

$A \rightarrow aA/a$

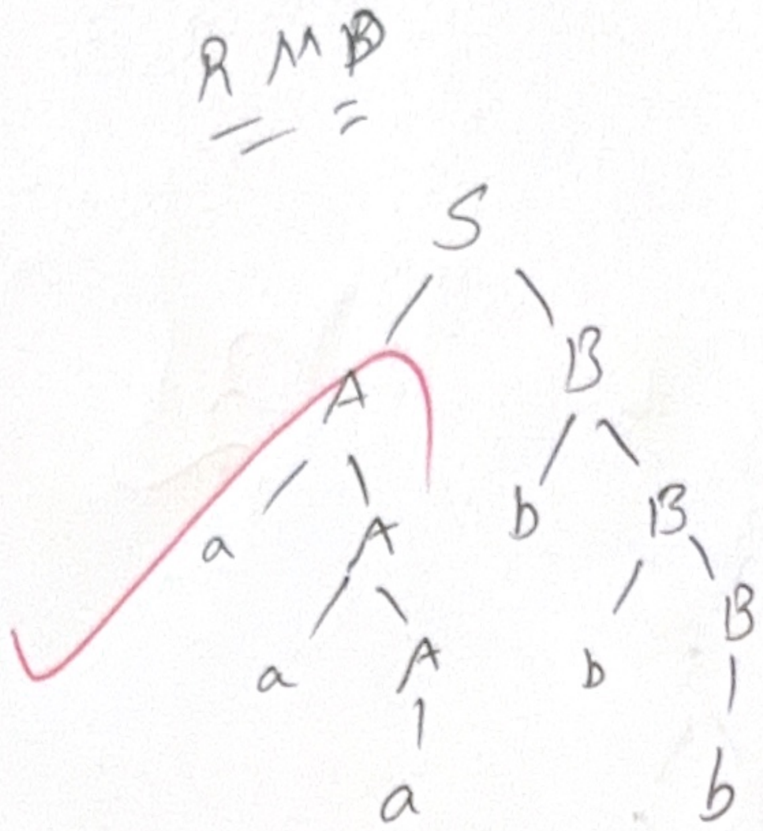
$B \rightarrow bB/b$

Input string = aaabbb

LMD



RMD



(14)

```
%{
#include<stdio.h>
%}

%%
[a-zA-Z]+
  [0-9]+
  [\t\n]
%%
```

parse expression

$E \rightarrow E+T$

$E \rightarrow T$

$T \rightarrow id$

$T \rightarrow num.$

$S \rightarrow AB$

$A \rightarrow aA/\varepsilon$

$B \rightarrow bB/c$.

Non-terminal Follow

First (S) = {a, b, c}

First (A) = {a, ε}

First (B) = {b, c}

Follow (S) = {$}

Follow (A) = {b, ε}

Follow (B) = {ε}

2

Left Recursion

$A \rightarrow A\alpha / \beta$.

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' / \varepsilon$

Left Factoring

$A \rightarrow \alpha\beta_1 / \alpha\beta_2$

$A \rightarrow \alpha A'$

$A' \rightarrow \beta_1 / \beta_2$

3

(18)

Input String = id + id * id.

$E \rightarrow TE'$

$E' \rightarrow +TE' / \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT / \varepsilon$

$F \rightarrow (E) / id$

first (E) = { , id}

first (E') = {+, ε}

first (T) = { , id}

first (T') = {*, ε}.

first F = { , id}

follow (E) = { $,)}

follow (E') = {$,)}

follow (T) = {+, $,)}

follow (T') = {+, *, $,)}

follow (F) = { *, +, $,)}.

Parse Table:

non-terminal	id	+	*	(	)	$
E	E→TE'			E→TE'		
E'		E'→+TE'			E'→TE'	
T	T→FT'			T→FT'		
T'		T'→ε	T'→ε		T'→ε	
F	F→id			F→(E)		

Stack implementation:

Stack	Input	output
$	id+id* id $	E→TE'
$E'T	id+id* id $	E→TE'
$E'T'F	id+id* id $	T→FT'
$E'T'id	id+id* id $	F→id
$E'	+id* id $	T'→ε
$E'T+	id* id $	E'→+TE'
$E'T	id* id $	—

2

id + id * id = input string

E -> E+E

E -> E*E

E -> id

Stack	input Buffer	Action
$	id+id*id $	shift
$id	+id* id $	Reduced by E -> id
$E	+id* id $	shift
$E+	id* id $	shift
$E+id	* id $	Reduced by E -> id
$E+E	* id $	REduce by E -> E+E
$E	* id $	shift
$E*	id $	shift
$E* id	$	Reduce by E -> id
$E*E	$	Reduce by E -> E*E
$E	$	Accepted.

(19)

$E \rightarrow E+E$ $E \rightarrow id$

$E \rightarrow E*E$

$E \rightarrow (E)$

Input string = id1 + id2 * id3

Stack	Input buffer	Actions
$	id + id * id $	shift
$ id	+ id * id $	Reduce by $E \rightarrow id$
$ E	+ id * id $	shift
$ E +	id * id $	shift
$ E + id	* id $	Reduce by $E \rightarrow id$
$ E + E	* id $	Reduce by $E \rightarrow E+E$
$ E	* id $	shift
$ E *	id $	shift
$ E * id	$	Reduce by $E \rightarrow id$
$ E * E	$	Reduce by $E \rightarrow E*E$
$ E	$	Accepted.

10

```
#include <stdio.h>

int main(){

    int a=10, b=20;

        int sum=a+b;

    Printf("%d", sum);

return 0;

}
```

Lexical analyzer: Source code converts into tokens

syntax analyzer: That tokens are converts into parse tree.

semantic: Verifies ~~Input~~ varaibles.

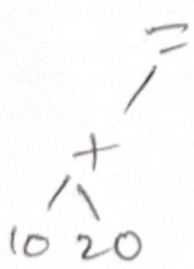
Intermediate code: Three address memory.

Code optimizer: It reduces the code.

code Generator: Converts into machine code.

b) sum = a+b.

sum = 10 + 20



c)

Sum = a + b.

S -> id = E

E -> E + T | T

T -> id / num

Input string = sum = a + b.